

SkillBase — Project Documentation & Technical Overview

A Pure Vanilla Web Platform for Peer-to-Peer Skill Matching & Direct Talent Discovery

Live URL:
https://skillbase24.vercel.app

Repository: github.com/satyamshukla-2412/SkillBase

Architecture: HTML5 / CSS3 / Vanilla JS (ES6+)

1. Executive Summary & Core Concept

SkillBase is a web-based skill-exchange and talent matching platform designed to connect people who **HAVE** valuable skills with individuals and organizations who **NEED** talent for projects, tasks, and creative collaborations. The platform eliminates intermediaries, recruitment agencies, and third-party fees by offering a direct, transparent match-making mechanism powered by deterministic keyword matching and geographic proximity scoring.

2. Target Personas & Platform Roles

Target Persona	Who They Are	Primary Objectives	Key Features Used
Skill Provider	Freelancers, developers, designers, content writers, skilled students, and domain specialists.	Showcase expertise, build a public portfolio, find matching project leads, and receive direct inquiries.	Profile manager, dynamic skill tag additions/removals, matches feed, received messages inbox.
Skill Seeker	Project owners, startup founders, community organizers, NGOs, and teams needing skilled help.	Post concrete project requirements, specify budgets, search qualified talent, and initiate contact.	Requirement posting, skill & city search filters, candidate match scoring, direct inquiry modal.

3. Technology Stack & Tools Used

Layer	Technology	Implementation Details
Markup (Structure)	HTML5 (Semantic)	Multi-page architecture with 7 distinct semantic views: <code>index.html</code> , <code>login.html</code> , <code>register.html</code> , <code>provider.html</code> , <code>seeker.html</code> , <code>matches.html</code> , and <code>profile.html</code> .
Styling (Aesthetics)	Vanilla CSS3	Zero external UI frameworks (No Bootstrap, No Tailwind). Uses native CSS Variables, CSS Grid, Flexbox, responsive media queries, and dual external/embedded fail-safe stylesheets.
Client Logic	Vanilla JavaScript (ES6+)	Modular separation of concerns across <code>storage.js</code> , <code>auth.js</code> , <code>main.js</code> , <code>matches.js</code> , <code>provider.js</code> , and <code>seeker.js</code> without heavy JS libraries.
Persistence	Browser LocalStorage	Client-side persistence for user registrations, active authentication sessions, requirements, and sent/received contact messages. Pre-seeded with 8 sample profiles.
Serverless Runtime	Node.js (HTTP / Lambda)	Universal endpoint (<code>app.js</code>) that serves all assets from an in-memory routing table with 100% reliability, MIME accuracy, and clean URL resolution.
Hosting & CI/CD	GitHub & Vercel	Automated deployments triggered directly from GitHub commits on the <code>main</code> branch to Vercel's global edge network.

4. Design Philosophy & Approach

The platform was intentionally crafted to feel like a **clean, practical, hand-designed website** rather than a generic AI-generated landing page template.

- Natural Aesthetics:** Uses a crisp light background (`#f9fafb`), dark charcoal text (`#1f2a37`), and a professional collegiate blue primary color (`#1a56db`).
- Modest Visual Weight:** Modest border-radii (4px to 6px), standard 1px solid borders, and clear typographic hierarchy without glowing neon gradients, glassmorphism, or 3D blobs.
- Zero Third-Party Assets:** Avoids external font scripts or icon packs that could break offline or behind restrictive networks.

5. Architecture & File Structure

```
skillbase/
├─ index.html      # Public homepage with Hero, 3 Benefits, How-It-Works, and About
├─ login.html      # Login portal with role switcher and 1-click test credentials
├─ register.html   # Dynamic registration with conditional Provider / Seeker inputs
├─ provider.html   # Skill Provider Dashboard: view/edit bio, manage skills, view inbox
├─ seeker.html     # Skill Seeker Dashboard: active requirement view, searches, sent log
├─ matches.html    # Skill Matching Engine: dual-mode provider and project search
├─ profile.html    # Detailed public profile view with direct contact trigger
├─ app.js          # Universal HTTP server and Vercel serverless request handler
├─ package.json    # Node metadata and endpoint declaration
├─ style.css       # Root stylesheet fallback
├─ css/
│ └─ style.css     # Core design system and responsive styles
└─ js/
  ├─ storage.js    # LocalStorage initialization, CRUD, and deterministic algorithm
  ├─ auth.js       # User registration, login, logout, and auth route protection
  ├─ main.js       # Navbar updates, mobile menu toggle, and global contact modal
  ├─ provider.js   # Provider dashboard logic (add/remove skills, edit bio)
  ├─ seeker.js     # Seeker dashboard logic (edit requirement, search providers)
  └─ matches.js    # Search execution, scoring calculation, and results rendering
```

6. How the Platform Works (User Flows)

1. Homepage Discovery

Visitors are greeted by the clean hero section ("*Connect Skills. Find Opportunities.*") with two clear pathways: **"I Have Skills"** (leading to Provider registration) and **"I Need Skills"** (leading to Seeker registration). Below the hero, three value propositions are highlighted: **Verified Skills**, **Smart Matching**, and **Direct Contact**.

2. Dynamic Registration & Authentication

On `register.html`, selecting **Skill Provider** dynamically opens inputs for primary skills, experience duration, location, and bio. Selecting **Skill Seeker** dynamically switches to required skill, project deliverables, location, and budget. Submitting validates input, ensures no duplicate emails, saves the user to `LocalStorage`, logs in automatically, and redirects to their dashboard.

3. Provider Dashboard Workflow

Providers can inspect their public card, add new skills on the fly (which immediately generates a removable badge), edit their bio and experience level via a clean modal, and view project inquiries sent to them by seekers.

4. Seeker Dashboard Workflow

Seekers view their active requirement (deliverable description, budget, and target city), edit requirement parameters at any time, perform direct candidate searches, and review the history of inquiries they have initiated.

7. Deterministic Skill-Matching Algorithm

SkillBase uses a transparent, deterministic matching algorithm coded in vanilla JavaScript (`js/storage.js` and `js/matches.js`):

Matching Stage	Condition	Score Awarded	Resulting Badge & Priority
Skill Keyword Match	Seeker requirement matches any Provider skill keyword (e.g., "Web Development").	+10 points	Skill Match
Location Bonus	Candidate location matches Seeker location (e.g., both in "Mumbai").	+5 points	Location Match
Combined Top Match	Both skill keyword and location criteria match simultaneously.	+15 points	Top Match (Skill + Location) (Ranked 1st)
General Browse	No specific search query entered (browsing directory).	+5 points (base)	Displays all available candidates sorted alphabetically.

Dual-Mode Discovery: On `matches.html`, users can toggle between "Find Skill Providers" (talent directory) and "Find Project Requirements" (open project opportunities).

8. Data Storage Architecture (LocalStorage Schema)

The platform runs completely client-side using three normalized LocalStorage tables:

```
1. 'skillbase_users'      -> Array of user objects (Providers & Seekers)
   Structure: { id, name, email, password, role, skills, experience, location, description, requiredSkill, bu

2. 'skillbase_current_user' -> Object representing the currently logged-in session user

3. 'skillbase_messages'  -> Array of inquiry messages
   Structure: { id, senderName, senderEmail, recipientId, recipientName, recipientRole, message, date }
```

9. Pre-Configured Demo Test Accounts

To enable immediate testing, the platform is pre-loaded with realistic accounts (password for all: password123):

Name	Role	Email	Skill / Requirement	Location
Rahul Sharma	Provider	rahul@example.com	HTML, CSS, JavaScript, React, Web Development	Mumbai
Priya Patel	Provider	priya@example.com	Figma, UI Design, Wireframing, Web Design	Pune
Aman Verma	Provider	aman@example.com	Python, SQL, Data Analysis, Machine Learning	Delhi
Sneha Roy	Provider	sneha@example.com	Content Writing, SEO, Technical Writing	Bangalore
TechFest Committee	Seeker	techfest@example.com	Required: Web Development (Budget: ₹8,000)	Mumbai
GreenCampus NGO	Seeker	greencampus@example.com	Required: UI Design (Budget: ₹4,000)	Pune

10. Deployment & Live Endpoints

- **Production Web Application:** <https://skillbase24.vercel.app>
- **GitHub Source Repository:** <https://github.com/satyamshukla-2412/SkillBase>
- **Direct Login Portal:** <https://skillbase24.vercel.app/login>
- **Direct Matching Portal:** <https://skillbase24.vercel.app/matches>